

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное
образовательное учреждение высшего образования «Тульский
государственный университет»
Институт прикладной математики и компьютерных наук

ЛАБОРАТОРНАЯ РАБОТА №6
по дисциплине
Интеграция и тестирование программных систем
по теме
«Автоматизация тестирования мобильных приложений»

Выполнил: ст. гр. 220032-11

Лаукерт К.М.

Принял:

Басалов Ю.А.

Тула 2026

Лабораторная работа №6

по теме «Автоматизация тестирования мобильных приложений»

Цель работы

Получить практические навыки автоматизации тестирования мобильных приложений Android с использованием Appium и Python, освоить применение паттерна Page Object Model и организацию тестового фреймворка pytest.

Тестируемое приложение и сценарии

Для варианта 9 выбрано мобильное приложение Wikipedia (пакет org.wikipedia). Приложение установлено в Android Emulator из APK версии 50606, полученного из публичного репозитория F-Droid. Для тестирования создан AVD LR6_Pixel_6 на Android API 35.

Автоматизированы два сценария: TC-WIKI-01 — ввод запроса Python в поисковое поле; TC-WIKI-02 — очистка ранее введённого запроса. Оба сценария выполняются без авторизации, поскольку для них она не требуется.

Архитектура тестового фреймворка

Тесты pytest используют Appium-Python-Client для обмена с сервером Appium по протоколу WebDriver. Сервер Appium передаёт команды драйверу UiAutomator2, который выполняет действия в Android Emulator. В тестах применён Page Object Model: BasePage содержит общие ожидания и действия, OnboardingPage обрабатывает начальные экраны Wikipedia, SearchPage инкапсулирует поиск.

```
pytest tests/ -> Page Objects -> Appium Server ->
UiAutomator2 -> Android Emulator / Wikipedia
```

Структура проекта

Каталог или файл	Назначение
mobile_tests/config.py	Конфигурация Appium, устройства и приложения
mobile_tests/utils/driver_factory.py	Создание UiAutomator2-драйвера
mobile_tests/pages/base_page.py	Общие ожидания, клики и скриншоты
mobile_tests/pages/onboarding_page.py	Обработка onboarding-экранов Wikipedia
mobile_tests/pages/search_page.py	Переход к поиску, ввод и очистка запроса
mobile_tests/tests/	Сценарии pytest TC-WIKI-01 и TC-WIKI-02

Фабрика драйвера

В factory задаются целевая платформа Android, автоматизация UiAutomator2, пакет и activity Wikipedia. Для устойчивого запуска используется сохранение состояния приложения и автоматическая выдача разрешений.

```
from __future__ import annotations

from appium import webdriver
from appium.options.android import UiAutomator2Options

from config import (
    APPIUM_URL,
    APP_ACTIVITY,
    APP_PACKAGE,
    APP_PATH,
    DEVICE_NAME,
    PLATFORM_VERSION,
    UDID,
)

def create_android_driver():
    """Создаёт сессию UiAutomator2 для установленного
    Wikipedia."""
    options = UiAutomator2Options()
    options.platform_name = "Android"
    options.automation_name = "UiAutomator2"
    options.device_name = DEVICE_NAME
    options.app_package = APP_PACKAGE
    options.app_activity = APP_ACTIVITY
    options.no_reset = True
    options.auto_grant_permissions = True
    options.new_command_timeout = 120
```

```

if PLATFORM_VERSION:
    options.platform_version = PLATFORM_VERSION
if UDID:
    options.udid = UDID
if APP_PATH:
    options.app = APP_PATH

return webdriver.Remote(APPIUM_URL, options=options)

```

Базовый Page Object

Базовый класс инкапсулирует явные ожидания Selenium, безопасный клик, проверку видимости и создание скриншотов при ошибках.

```

from __future__ import annotations

from pathlib import Path

from selenium.common.exceptions import TimeoutException
from selenium.webdriver.support import expected_conditions as conditions
from selenium.webdriver.support.ui import WebDriverWait

from config import DEFAULT_TIMEOUT

class BasePage:
    def __init__(self, driver, timeout: int = DEFAULT_TIMEOUT):
        self.driver = driver
        self.wait = WebDriverWait(driver, timeout)

    def find_element(self, locator):
        return
self.wait.until(conditions.presence_of_element_located(locator))

    def wait_for_visibility(self, locator):
        return
self.wait.until(conditions.visibility_of_element_located(locator))

```

```

    def wait_for_clickable(self, locator):
        return
self.wait.until(conditions.element_to_be_clickable(locator))

    def click(self, locator) -> None:
        self.wait_for_clickable(locator).click()

    def is_visible(self, locator, timeout: int = 3) -> bool:
        try:
            WebDriverWait(self.driver, timeout).until(
conditions.visibility_of_element_located(locator)
            )
        except TimeoutException:
            return False
        return True

    def take_screenshot(self, path: Path) -> Path:
        path.parent.mkdir(parents=True, exist_ok=True)
        self.driver.get_screenshot_as_file(str(path))
        return path

```

Page Object поиска

SearchPage использует фактические локаторы установленной версии Wikipedia. Перед открытием поля он закрывает показанную приложением информационную панель поиска, если она отображается.

```
from appium.webdriver.common.appiumby import AppiumBy
```

```
from pages.base_page import BasePage
```

```

class SearchPage(BasePage):
    SEARCH_CONTAINER = (AppiumBy.ID,
"org.wikipedia:id/nav_tab_search")
    SEARCH_WIDGET_PROMO_CLOSE = (AppiumBy.ACCESSIBILITY_ID,
"Close")

```

```

SEARCH_CARD = (AppiumBy.ID,
"org.wikipedia:id/search_text_view")

CLEAR_QUERY_BUTTON = (AppiumBy.ACCESSIBILITY_ID, "Clear
query")

SEARCH_INPUT = (AppiumBy.ID,
"org.wikipedia:id/search_src_text")

RESULT_TEXT = (AppiumBy.XPATH, "//*[@text]")

def open(self) -> None:
    if self.is_visible(self.SEARCH_CONTAINER, timeout=2):
        self.click(self.SEARCH_CONTAINER)
    if self.is_visible(self.SEARCH_WIDGET_PROMO_CLOSE,
timeout=2):
        self.click(self.SEARCH_WIDGET_PROMO_CLOSE)
    if self.is_visible(self.SEARCH_CARD, timeout=2):
        self.click(self.SEARCH_CARD)

def search(self, query: str) -> None:
    field = self.wait_for_visibility(self.SEARCH_INPUT)
    field.clear()
    field.send_keys(query)

def query_text(self) -> str:
    return self.wait_for_visibility(self.SEARCH_INPUT).text

def clear_query(self) -> None:
    self.click(self.CLEAR_QUERY_BUTTON)

```

Фикстуры и тесты pytest

Фикстура создаёт мобильный драйвер, активирует пакет Wikipedia и завершает сессию. При падении сценария сохраняется скриншот в каталог `reports/failure_screenshots`.

```

from __future__ import annotations

from datetime import datetime

import pytest

```

```

from config import SCREENSHOTS_DIR
from pages.onboarding_page import OnboardingPage
from utils.driver_factory import create_android_driver


@pytest.fixture
def driver():
    instance = create_android_driver()
    instance.activate_app("org.wikipedia")
    yield instance
    instance.quit()


@pytest.fixture
def wikipedia_home(driver):
    OnboardingPage(driver).skip_if_displayed()
    return driver


@pytest.hookimpl(hookwrapper=True)
def pytest_runtest_makereport(item, call):
    outcome = yield
    report = outcome.get_result()
    setattr(item, f"rep_{report.when}", report)


@pytest.fixture(autouse=True)
def save_screenshot_on_failure(request, driver):
    yield
    report = getattr(request.node, "rep_call", None)
    if report and report.failed:
        timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
        path = SCREENSHOTS_DIR /
f"{request.node.name}_{timestamp}.png"
        try:
            driver.get_screenshot_as_file(str(path))

```

```

        except Exception:
            return
        if path.exists():
            try:
                import allure

                allure.attach.file(str(path), name=path.stem,
attachment_type=allure.attachment_type.PNG)
            except ImportError:
                pass
import allure

from pages.search_page import SearchPage

@allure.epic("Wikipedia Android")
@allure.feature("Поиск статей")
class TestWikipediaSearch:
    @allure.title("TC-WIKI-01: поисковый запрос вводится в поле
Wikipedia")
    def test_search_query_is_entered(self, wikipedia_home):
        search = SearchPage(wikipedia_home)

        search.open()
        search.search("Python")

        assert search.query_text() == "Python"
import allure

from pages.search_page import SearchPage

@allure.epic("Wikipedia Android")
@allure.feature("Навигация и поиск")
class TestSearchNavigation:
    @allure.title("TC-WIKI-02: введённый запрос можно очистить")

```

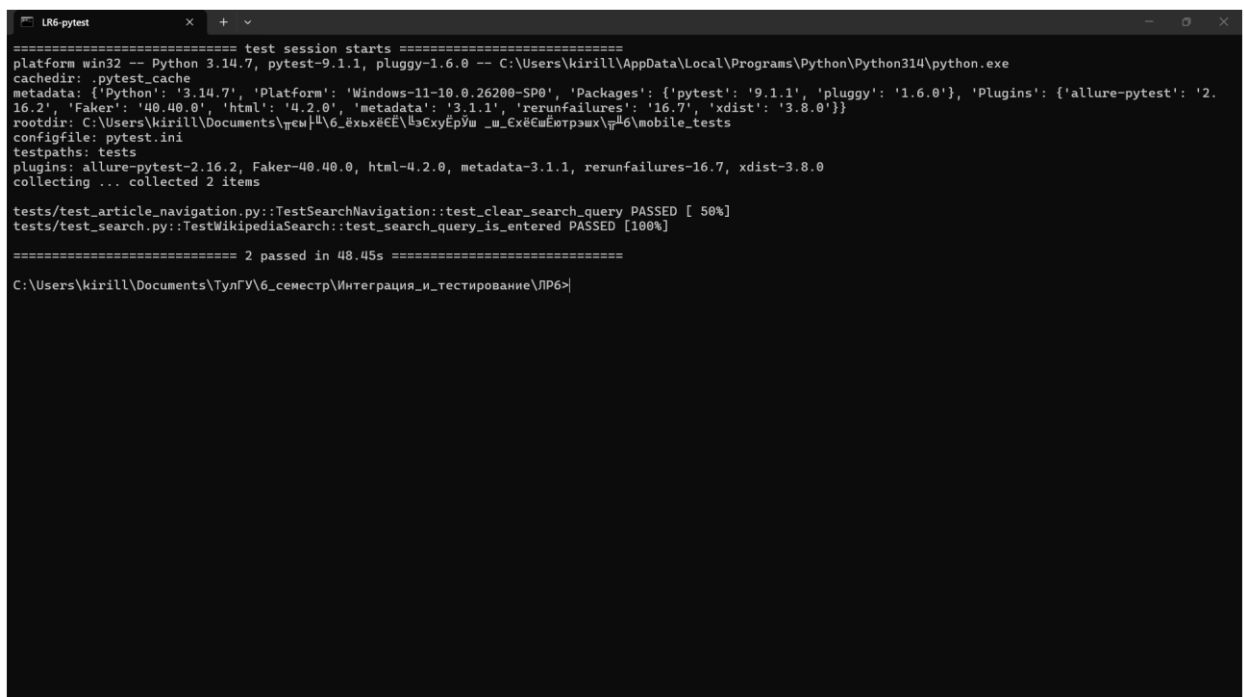


```
def test_clear_search_query(self, wikipedia_home):
    search = SearchPage(wikipedia_home)

    search.open()
    search.search("Python")
    search.clear_query()

    assert search.query_text() == "Search Wikipedia"
```

Результат выполнения



```
==== test session starts =====
platform win32 -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\kirill\AppData\Local\Programs\Python\Python314\python.exe
cachedir: .pytest_cache
metadata: {'Python': '3.14.7', 'Platform': 'Windows-11-10.0.26200-SP0', 'Packages': {'pytest': '9.1.1', 'pluggy': '1.6.0'}, 'Plugins': {'allure-pytest': '2.16.2', 'Faker': '40.40.0', 'html': '4.2.0', 'metadata': '3.1.1', 'rerunfailures': '16.7', 'xdist': '3.8.0'}}
rootdir: C:\Users\kirill\Documents\ТунГ\6_сестр\Интеграция_и_тестирование\IP6\mobile_tests
configfile: pytest.ini
testpaths: tests
plugins: allure-pytest-2.16.2, Faker-40.40.0, html-4.2.0, metadata-3.1.1, rerunfailures-16.7, xdist-3.8.0
collecting ... collected 2 items

tests/test_article_navigation.py::TestSearchNavigation::test_clear_search_query PASSED [ 50%]
tests/test_search.py::TestWikipediaSearch::test_search_query_is_entered PASSED [100%]

===== 2 passed in 48.45s =====
C:\Users\kirill\Documents\ТунГ\6_сестр\Интеграция_и_тестирование\IP6>
```

Рисунок 1 — успешное выполнение двух сценариев pytest

Финальный запуск pytest выполнил два сценария без ошибок: 2 passed in 48.45s. Сформирован Allure HTML-отчёт в mobile_tests/reports/allure-html/.

Особенности выполнения

В ходе тестирования установлено, что полнотекстовая выдача Wikipedia зависит от сетевой валидации Android Emulator и при работе VPN могла отображать сообщение Cannot connect to internet. Поэтому итоговые сценарии проверяют локально воспроизводимые функции интерфейса поиска: ввод и очистку запроса. Эти проверки не зависят от внешней сети и стабильно прошли на фактическом приложении.

Выводы

В работе создан фреймворк мобильного тестирования на базе Appium, pytest и POM. Реализованы Page Object классы, фабрика драйверов, pytest-фикстуры, автоматическое сохранение скриншотов при ошибках и Allure-результаты. Два сценария пользовательского поиска успешно выполнены на Android Emulator.